

TEXT RECOGNITION USING OCR

Project Title

Text Recognition from Images using Optical Character Recognition (OCR)

Submitted By

Ayisha Sana Parveena

Objective

The objective of this project is to develop a system that can automatically extract text from images using Optical Character Recognition (OCR). The project utilizes image preprocessing techniques and OCR technology to recognize and convert image-based text into machine-readable text.

Introduction

Optical Character Recognition (OCR) is a technology used to identify and extract text from images, scanned documents, and photographs.

OCR plays an important role in digitizing printed content and making it searchable and editable.

In this project, OpenCV is used for image preprocessing, while Tesseract OCR is used for text extraction.

Technologies Used

- Python
- OpenCV
- Pytesseract
- NumPy
- Tesseract OCR Engine

Libraries Used

OpenCV

OpenCV is used for image processing operations such as grayscale conversion, blurring, and thresholding.

Pytesseract

Pytesseract acts as a Python wrapper for Google's Tesseract OCR Engine and helps extract text from images.

NumPy

NumPy supports numerical operations and image array manipulation.

Methodology

Step 1: Image Loading

The input image is loaded into the program using OpenCV.

Step 2: Image Preprocessing

The image undergoes preprocessing to improve OCR accuracy:

- Grayscale Conversion
- Gaussian Blur
- Adaptive Thresholding

Step 3: Text Extraction

The processed image is passed to Tesseract OCR for text recognition.

Step 4: Output Generation

The extracted text is displayed in the terminal and the processed image is shown to the user.

Code Workflow

Input Image



Grayscale Conversion



Gaussian Blur



Adaptive Thresholding



OCR Processing



Extracted Text Output

Source Code

```
import cv2

import pytesseract

pytesseract.pytesseract.tesseract_cmd = r"C:\Program
Files\Tesseract-OCR\tesseract.exe"

image = cv2.imread("sample.png")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

blur = cv2.GaussianBlur(gray, (5, 5), 0)

thresh = cv2.adaptiveThreshold(
    blur,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)

text = pytesseract.image_to_string(thresh, config="--psm 6")
```

```
print("Extracted Text:")
```

```
print(text)
```

```
cv2.imshow("Processed Image", thresh)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Sample Input

Image containing the following text:

HELLO WORLD

AI Internship Project

DecodeLabs

Sample Output

Extracted Text:

HELLO WORLD

AI Internship Project

DecodeLabs

Applications

- Document Digitization

- Automated Data Entry
- Invoice Processing
- License Plate Recognition
- Educational Content Digitization
- Text Extraction from Images

Advantages

- Fast text extraction
- Reduces manual typing effort
- Improves productivity
- Supports digital document management.

Future Enhancements

- Handwritten Text Recognition
- Multi-language OCR Support
- Real-time Camera OCR
- PDF Text Extraction
- OCR Web Application Development.

Conclusion

The Text Recognition using OCR project successfully extracts text from images using image preprocessing and OCR techniques. The project demonstrates the practical application of Computer Vision

and Artificial Intelligence concepts in automating text recognition tasks.

References

1. OpenCV Documentation
2. Tesseract OCR Documentation
3. Python Official Documentation.